```
$ rspec
..**

Pending: (Failures listed here are expected and do not affect your suite's ↩
status)

  1) A cup of coffee with milk is light in color
     # Color not implemented yet
     Failure/Error: expect(coffee.color).to be(:light)

     NoMethodError:
       undefined method `color' for #<Coffee:0x007f83b1199a88 ↩
       @ingredients=[:milk]>
     # ./spec/coffee_spec.rb:36:in `block (3 levels) in <top (required)>'

  2) A cup of coffee with milk is cooler than 200 degrees Fahrenheit
     # Temperature not implemented yet
     Failure/Error: expect(coffee.temperature).to be < 200.0

     NoMethodError:
       undefined method `temperature' for #<Coffee:0x007f83b11984d0 ↩
       @ingredients=[:milk]>
     # ./spec/coffee_spec.rb:41:in `block (3 levels) in <top (required)>'

Finished in 0.00161 seconds (files took 0.07898 seconds to load)
4 examples, 0 failures, 2 pending
```

Of course, you could just comment out the examples instead of marking them as pending. But unlike commented-out code, pending examples still run and report their failures—which means you can use this information to drive your implementation.

Define an inspect Method for Clearer Test Output



Some failure messages—such as the NoMethodError exceptions printed for these pending coffee specs—include string representations of your objects. Ruby (and RSpec) generate this string by calling inspect on each object.

If a particular class doesn't define an inspect method, the resulting string will be something like #<Coffee:0x007f83b11984d0 @ingredients=[:milk]>. To make the output a little more programmer-friendly, we recommend defining this method to print a nice, readable string such as #<Coffee (with milk)>.

Completing Work in Progress

One of the nice things about marking examples as pending is that RSpec will let you know when they start passing.